

 PROYECTO DE ANÁLISIS Y DISEÑO DE ALGORITMOS

Planificador de Horarios Universitarios

Comparativa de técnicas algorítmicas para la asignación óptima de
materias, salones y horarios

 Miguel Ángel Guarnizo Salazar
Juan Sebastián Villa Rodas
Tomás Buriticá Jaramillo

 5 algoritmos implementados

| El Problema

¿Por qué la planificación de horarios es un desafío complejo?



Relevancia

La planificación de horarios universitarios es un problema **NP-duro** que afecta diariamente a miles de instituciones educativas.

Una mala asignación genera:

- Conflictos de horario para profesores y estudiantes
- Desperdicio de recursos físicos (salones ociosos)
- Insatisfacción por no respetar preferencias



Objetivo

Asignar materias (con profesor, grupo, alumnos y horas requeridas) a salones disponibles (con capacidad limitada) en horarios específicos, **optimizando tres objetivos simultáneamente:**

- 1 Evitar conflictos de horario
- 2 Maximizar el uso de salones
- 3 Respetar preferencias de horario

| 3 Subproblemas

Los desafíos que el planificador debe resolver



SP1

Evitar Conflictos

Garantizar que no existan superposiciones imposibles entre profesores, salones y grupos.

- ✓ Un profesor no puede dar dos clases al mismo tiempo
- ✓ Un salón no puede albergar dos materias simultáneamente
- ✓ Respetar horarios bloqueados de cada profesor



SP2

Maximizar Salones

Optimizar la utilización de los recursos físicos disponibles de manera eficiente.

- ✓ Minimizar salones ociosos
- ✓ Uso eficiente de capacidad (ni muy grande ni muy pequeño)
- ✓ Distribución uniforme de la carga



SP3

Preferencias

Satisfacer las preferencias de los profesores cuando sea posible.

- ✓ Horarios preferidos con prioridad (alta, media, baja)
- ✓ Puntaje de satisfacción basado en preferencias cumplidas
- ✓ Ventanas horarias preferidas por materia

| 5 Técnicas Algorítmicas

Estrategias implementadas para resolver el problema de optimización



Fuerza Bruta

Exploración exhaustiva de todas las combinaciones posibles.



Recursivo

División del problema en subproblemas más pequeños.



Greedy

Selección de la opción óptima en cada paso local.



Backtracking

Retroceso con poda para explorar soluciones parciales.



Divide y Vencerás

Descomposición recursiva en subproblemas independientes.



Exploración completa



Heurístico rápido



Mejor rendimiento

Casos de Prueba

Tres escalas para evaluar el comportamiento de los algoritmos

Caso Pequeño

3 materias

 3 materias

 2 salones

 2 grupos

Escenario básico para validar correctitud

Caso Mediano

8 materias

 8 materias

 4 salones

 4 grupos

Escenario intermedio para medir escalabilidad

Caso Grande

15 materias

 15 materias

 6 salones

 6 grupos

Escenario realista para evaluar rendimiento

 Los casos de prueba aumentan progresivamente en complejidad para evaluar cómo escala cada algoritmo

| Resultados

Tiempos de ejecución (en segundos) para cada algoritmo y caso de prueba

Algoritmo	Caso Pequeño(3 materias)	Caso Mediano(8 materias)	Caso Grande(15 materias)
Fuerza Bruta	0.0093	0.0260	0.0994
Recursivo	0.0336	0.0210	0.0906
Greedy	0.0119	0.0236	0.0818
Backtracking	0.0074	0.0167	0.0730
Divide y Vencerás	0.0103	0.0131	0.0172

☐ Mejor tiempo por caso ☐ Mejor algoritmo overall

Mejor en Caso Pequeño

Backtracking

0.0074s

Mejor en Caso Mediano

Divide y Vencerás

0.0131s

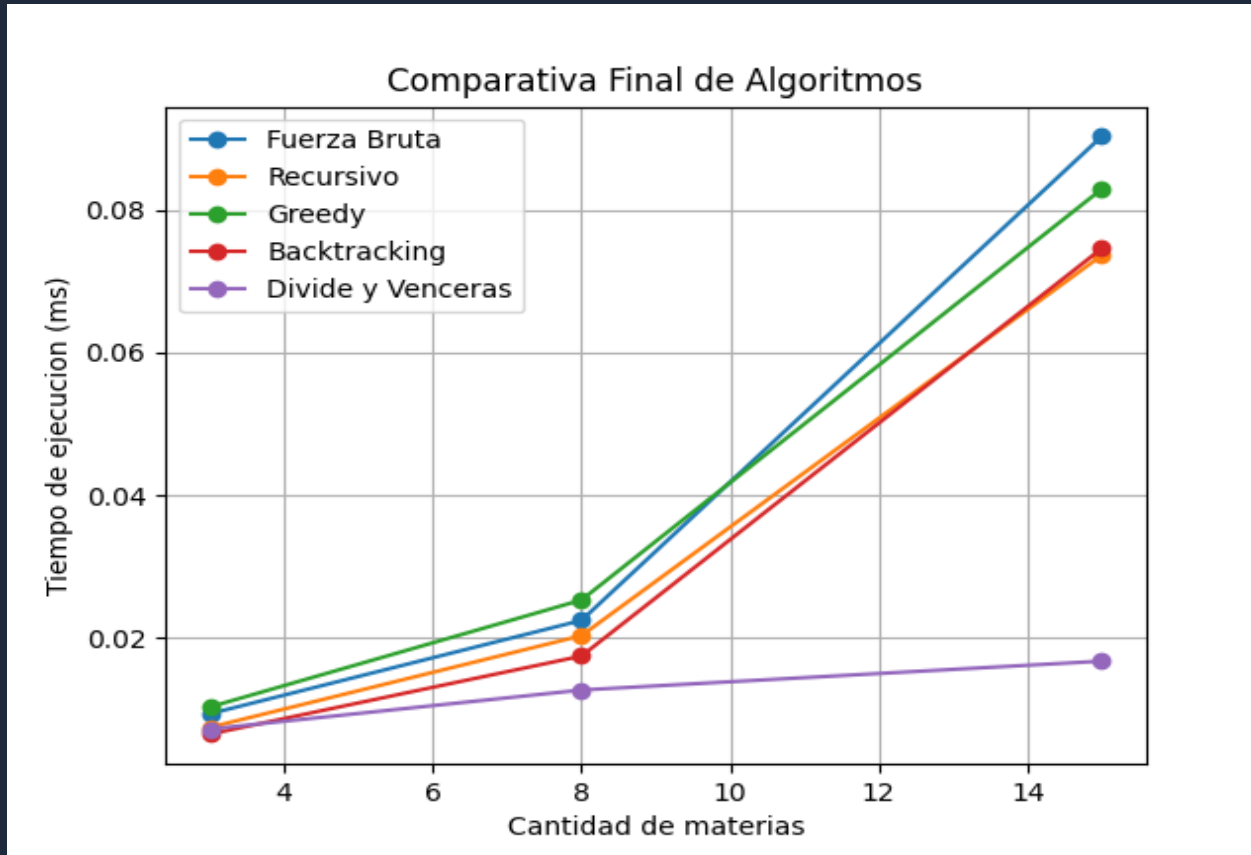
Mejor en Caso Grande

Divide y Vencerás

0.0172s

Comparación Visual

Rendimiento de los 5 algoritmos en los 3 casos de prueba



Fuerza Bruta

0.0994s

caso grande

Recursivo

0.0906s

caso grande

Greedy

0.0818s

caso grande

Backtracking

0.0730s

caso grande

Divide y Venceras

0.0172s

caso grande

| Análisis de Rendimiento

Hallazgos clave de la comparativa



Ganador: Divide y Vencerás

Es el **algoritmo más eficiente** en los 3 casos. En el caso grande, es **4.4x más rápido** que Backtracking (el segundo mejor) y **5.9x más rápido** que Fuerza Bruta.



Segundo: Backtracking

Muestra **buen rendimiento** consistente. Su estrategia de poda reduce significativamente el espacio de búsqueda, manteniendo tiempos competitivos.



Intermedio: Greedy

Rendimiento **aceptable** pero no óptimo. Como heurístico, sacrifica optimalidad por velocidad, siendo útil cuando el tiempo es crítico.



Peor Escalabilidad

Fuerza Bruta y Recursivo escalan peor al aumentar el tamaño. Fuerza Bruta pasa de 0.0093s a 0.0994s (10.7x), mientras que Recursivo mejora en casos medianos pero empeora en grandes.



Conclusión intermedia: La descomposición recursiva de Divide y Vencerás ofrece la mejor relación entre calidad de solución y tiempo de ejecución.

| Conclusiones

Reflexiones finales del proyecto

1 Mejor Algoritmo

Para problemas de planificación de horarios, **Divide y Vencerás** ofrece la mejor relación entre calidad de solución y tiempo de ejecución, siendo consistentemente el más rápido en todos los escenarios.

2 Elección Contextual

La elección del algoritmo depende del **tamaño del problema** y de si se prioriza la optimalidad vs la velocidad. No existe un único algoritmo perfecto para todos los escenarios.

3 Escalabilidad

Los algoritmos con **complejidad exponencial** (Fuerza Bruta, Recursivo) se vuelven inviables para instancias grandes, mientras que los de complejidad logarítmica (Divide y Vencerás) mantienen tiempos aceptables.

4 Trade-off Óptimo

El proyecto demuestra que **la descomposición inteligente** del problema (Divide y Vencerás) supera a tanto la fuerza bruta como a las heurísticas simples, ofreciendo el mejor trade-off entre calidad y eficiencia.



Gracias

Proyecto de Análisis y Diseño de Algoritmos
Comparativa de técnicas para planificación de horarios
universitarios

5

Algoritmos

3

Casos de Prueba

3

Subproblemas



github.com/Miguel-Angel-Salazar